Ders 6 Çalışma Özeti

Ders 6 Çalışma Özeti

Genel Konular

- Asıl ders yaklaşık 00:07'de başlar.
- Register transferi ve ortak veri yolu (bus)
  - Dersin hedefi: veri yolu birimi (datapath unit) ve kontrol birimi (control unit) ile bir CPU tasarlamak.
  - Ayrılmış çoklayıcı (dedicated multiplexer) yapısı.
  - Tek veri yolu (single bus) yapısı.
  - Üç durumlu tampon (three-state buffer) ve üç durumlu veri yolu (three-state bus).
- Komut kümesi mimarisi (Instruction Set Architecture, ISA)
  - RISC ve CISC.
  - Komut biçimi: opcode, adres alanı, mod alanı.
  - Komut çevrimi: IF, ID, OF, EX, WB.
  - Programcının erişebildiği register'lar, PC, IR, durum register'ı, yığın işaretçisi.
  - Açık (explicit) ve örtük (implied) adres.
  - Üç, iki, bir ve sıfır adresli komutlar.
  - Yığın (stack) işlemleri: push ve pop.
  - Adresleme modları.
  - Veri transfer, veri işleme (aritmetik, lojik, kaydırma) ve program kontrol komutları.
  - Bağımsız G/Ç (isolated I/O) ve belleğe eşlenmiş G/Ç (memory-mapped I/O).
  - Koşullu dallanma komutları ve durum bitleri; işaretli ve işaretsiz karşılaştırma örnekleri.

Hocanın Özellikle Vurguladığı Kısımlar

- Üç durumlu veri yolu konusu önemli. "Bunu anlamanızı özellikle istiyorum."
  - Bundan sonraki CPU tasarımında bu yapı kullanılacak.
- Üç durumlu veri yolunda aynı anda yalnız bir tamponun enable olmasını garanti edin.
  - İki register aynı anda veri yoluna farklı değer koyarsa devre yanar.
- Bu ders 8086 assembly dersi değildir.
  - Amaç tek bir işlemciyi değil, genel mimari kavramları öğrenmektir. Böylece yeni bir işlemciye kolay geçersiniz.
- Aritmetik kaydırmada taşma yalnız sola kaydırmada olur. Sağa kaydırmada taşma olmaz, yalnız işaret biti korunur.
- Negatif sayıyı yazarken yalnız en anlamlı biti 1 yapmayın.
  - Önce pozitif değeri yazın, sonra ikiye tümleyenini alın. Hocanın en çok gördüğü hatalardan biri.
- Koşullu dallanma tablosunda C sütunu aslında ödünç (borrow) bitidir. Borrow, carry'nin tümleyenidir.
- Koşullu dallanma tablosunu ezberlemek değil, nasıl oluştuğunu anlamak gerekir.
- Kitabın ISA bölümünün sonundaki soruları çözün.
- Bilimsel hazırlık öğrencileri assembly bilmediği için adresleme modlarına ayrıca çalışmalı.

Kısa Tekrar Notları

- Ayrılmış çoklayıcı: Her register'ın önünde kendi çoklayıcısı vardır. Esnektir ama pahalıdır.

- Tek veri yolu: Bütün register'lar için tek bir çoklayıcı vardır. Ucuzdur ama aynı anda yalnız bir kaynak aktarılabilir.
- Üç durumlu tampon: $E = 0$ iken çıkış High-Z, $E = 1$ iken çıkış = giriş.
- Üç durumlu veri yolu: Çoklayıcı yoktur. Register çıkışları üç durumlu tamponlarla ortak hatta bağlanır. Enable'ları bir decoder seçer.
- Komut çevrimi: Instruction Fetch (IF) → Instruction Decode (ID) → Operand Fetch (OF) → Execute (EX) → Write Back (WB) → PC güncellenir.
- Adresleme modlarında etkin adres (EA) ve AC'ye yüklenen değer (örnekte komut 250-251'de, adres alanı 500, PC = 252, R1 = 400, XR = 400):

Mod	Etkin adres	AC
Doğrudan (direct)	500	800
Anlık (immediate)	251	500
Dolaylı (indirect)	800	300
PC'ye göreli (relative)	752	600
İndisli (indexed)	900	200
Register	-	400
Register dolaylı	400	700

- RISC: az ve basit komut, az adresleme modu, sabit komut uzunluğu, belleğe yalnız load/store ile erişim, kablolu (hardwired) kontrol, hızlı.
- CISC: çok ve karmaşık komut, çok adresleme modu, değişken komut uzunluğu, işlemlerde doğrudan bellek operandı, mikroprogramlı kontrol.
- Lojik kaydırma: dışarıdan giren bit her zaman 0. Dışarı çıkan bit carry'ye gider.
- Sağa kaydırma ≈ 2'ye bölme (aşağı yuvarlama), sola kaydırma = 2 ile çarpma.
- Aritmetik sola kaydırmada taşma: $V = R_7 \oplus R_6$ (8 bitlik register, kaydırmadan önce).
- CMP: $A - B$ yapar, sonucu yazmaz, yalnız durum bitlerini günceller. TEST: $A \cdot B$ (AND) yapar, yalnız durum bitlerini günceller.
- İşaretsiz karşılaştırmada C (borrow) ve Z, işaretli karşılaştırmada N, V ve Z kullanılır.

Detaylı Açıklamalar

Dersin hedefi: CPU'nun iki bloğu

- Veri yolu birimi (datapath unit): aritmetik lojik birim (ALU), kaydırma birimi (shift unit), register dosyası (register file) ve bunları bağlayan iç veri yolu (internal bus).
- Kontrol birimi (control unit): ALU'ya ve veri yoluna hangi işlemin yapılacağını söyleyen kontrol işaretlerini gönderir.
- Veri yolu birimi işlemden sonra durum bitlerini (overflow, zero, negative, carry) kontrol birimine bildirir. Kontrol birimi sonraki adıma buna göre karar verir.
- Register'a yükleme saat girişinin önüne konan bir kapı ile yapılmaz. Yükleme denetimi D flip-flop'un D girişinin önüne konur. (Önceki haftalarda anlatıldı.)

Ayrılmış çoklayıcı (dedicated multiplexer)

- Örnekte 3 register var: R_0, R_1, R_2.
- Her register iki farklı kaynaktan yüklenebilir. Bu yüzden her register'ın önünde bir 2→1 çoklayıcı vardır.
  - R_0'ın çoklayıcısı: giriş 1 = R_1, giriş 0 = R_2.
- Her çoklayıcının bir seçme ucu (S) ve her register'ın bir yükleme ucu (L) vardır.
- Avantaj: Aynı anda farklı kaynaklardan farklı hedeflere transfer yapılabilir. Ör. $R_0 \leftarrow R_1$ ve $R_1 \leftarrow R_0$ aynı anda olur.
- Dezavantaj: Register sayısı 15-20 olursa her register'ın önüne büyük (16→1, 32→1) çoklayıcı gerekir. Çok pahalıdır, tercih edilmez.

Tek veri yolu (single bus)

- Bütün register çıkışları tek bir çoklayıcıya girer. Çoklayıcı çıkışı bütün register'ların girişine gider.

- 3 register için 4→1 çoklayıcı kullanılır (bir girişi boş). İki seçme ucu vardır: S_1S_0.
- Hangi register'ın yükleneceğini yükleme uçları ($L_2L_1L_0$) belirler.
- Örnekler:
  - $R_0 \leftarrow R_2$: $S_1S_0 = 10$, $L_2L_1L_0 = 001$.
  - $R_0 \leftarrow R_1$ ve $R_2 \leftarrow R_1$ (aynı kaynak): $S_1S_0 = 01$, $L_2L_1L_0 = 101$. Tek adımda yapılabilir.
  - $R_0 \leftarrow R_1$ ve $R_1 \leftarrow R_0$ (iki farklı kaynak): Tek veri yolunda aynı anda yapılamaz. Veri yolu bir anda yalnız bir register'ın içeriğini taşır. İki ayrı saat periyoduna bölmek de takası sağlamaz: önce $R_0 \leftarrow R_1$ yapılırsa R_0'ın eski değeri kaybolur. Takas için geçici bir alan gerekir.

Üç durumlu tampon ve veri yolu

- Üç durumlu tampon doğruluk tablosu:

E	Giriş	Çıkış
0	X	High-Z
1	0	0
1	1	1

- High-Z (yüksek empedans): Çıkış devreden kopmuş gibi davranır.
- Normal lojik kapıların çıkışları birbirine bağlanamaz. Biri 1 (+5 V), diğeri 0 üretirse yüksek akım akar ve kapı yanar.
- İki üç durumlu tamponun çıkışı birbirine bağlanabilir. Şart: Aynı anda en fazla birinin E girişi 1 olmalı.
  - İkisi birden enable ve girişleri farklı ise yine kısa devre olur.
- Çift yönlü (bidirectional) register:
  - Register çıkışına (Q çıkışları) bir üç durumlu tampon konur.
  - Tamponun çıkışı veri yoluna, veri yolu da bütün register'ların girişine bağlanır.
  - $E_i = 1$ ise R_i veri yoluna veri koyar. Bu veri, L ucu 1 olan register'lara yüklenir.
- Enable uçlarından yalnız birinin 1 olmasını bir decoder sağlar.
- Üç durumlu veri yolunda çoklayıcı yoktur. Çoklayıcının seçme uçlarının yerini enable uçları alır.
- Öğrenci soruları:
  - "Enable 1 iken register değerini korumak için load da açık olmalı mı?" Hayır. Tampon yalnız çıkışı veri yoluna verir. Register değerini korur.
  - "L_0 ve E_0 aynı anda 1 olursa?" R_0 kendi değerini kendine yeniden yükler. Yapı saatle eşzamanlı çalışır, sonuç bir sonraki periyotta görülür.

Komut kümesi mimarisi (ISA)

- Yüksek seviyeli dil → derleyici → assembly → assembler → makine kodu → işlemci.
- Bir mimari önce komut kümesiyle tanımlanır. Sonra bu kümeyi çalıştıracak donanım tasarlanır.
- RISC ve CISC karşılaştırması:

Özellik	RISC	CISC
Komutlar	Az, basit	Çok, karmaşık (çarpma, bölme vb.)
Adresleme modu	Az	Çok
Komut uzunluğu	Sabit	Değişken
Bellek erişimi	Yalnız load/store	İşlemlerde doğrudan bellek operandı
Kontrol birimi	Kablolu (hardwired), hızlı	Mikroprogramlı, esnek, yavaş
Örnek	-	8086

Komut biçimi

- Opcode (işlem kodu): Komutun ne yaptığını söyler (transfer, toplama, artırma...).
- Adres alanı: Operandın adresini verir. Bir, iki ya da üç adres alanı olabilir.
- Mod alanı: Adresleme modunu söyler.
- Gerekirse başka alanlar.

Komut çevrimi

1. Instruction Fetch (IF): Komut bellekten komut register'ına (IR) getirilir.
2. Instruction Decode (ID): Opcode ve adresleme modu çözülür.
3. Operand Fetch (OF): Operand (gerekirse bellekten) getirilir.
4. Execute (EX): İşlem yürütülür.
5. Write Back (WB): Sonuç register'a ya da belleğe yazılır.
6. PC sonraki komutu gösterecek biçimde güncellenir, çevrim başa döner.

- Bu adımlar pipeline konusunda yeniden kullanılacak.

Tasarlanacak basit mimarinin register'ları

- 16 register: R_0-R_7 programcının erişebildiği register'lar, R_8-R_{15} mikro işlemler için iç register'lar.
- Komut register'ı (IR): Bellekten getirilen komutu tutar, çözme işlemine girer.
- Program sayacı (PC): Yürütülecek komutun adresini tutar. Her komuttan sonra artırılır, bu yüzden "sayaç" denir.
- Pipeline register'ları, durum register'ı (C, V, N, Z bayrakları), yığın işaretçisi (SP).

Açık ve örtük adres

- Açık (explicit) adres: Adres komutun adres alanında verilir.
- Örtük (implied) adres: Operandın yeri opcode'dan anlaşılır. Ör. "load to accumulator" komutunda hedef her zaman akümülatördür.

Adres sayısına göre komutlar: $X = (A + B)(C + D)$

- Üç adresli:

```
ADD T1, A, B     ; M[T1] <- M[A] + M[B]
ADD T2, C, D     ; M[T2] <- M[C] + M[D]
MUL X, T1, T2    ; M[X]  <- M[T1] * M[T2]
```

- Avantaj: kısa program. Dezavantaj: komut çok bit gerektirir.
- İki adresli: Bir adres hem kaynak hem hedeftir.

```
MOVE T1, A       ; M[T1] <- M[A]
ADD  T1, B       ; M[T1] <- M[T1] + M[B]
MOVE X, C        ; M[X]  <- M[C]
ADD  X, D        ; M[X]  <- M[X] + M[D]
MUL  X, T1       ; M[X]  <- M[X] * M[T1]
```

- Hocanın sorusu: A neden önce T1'e taşındı? Cevap: $M[A]$'nın değerini bozmamak için. ADD sonucu ilk operandın üstüne yazar.
- Bir adresli: Akümülatör (ACC) örtük operanddır.

```
LD  A     ; ACC <- M[A]
ADD B     ; ACC <- ACC + M[B]
ST  X     ; M[X] <- ACC
LD  C     ; ACC <- M[C]
ADD D     ; ACC <- ACC + M[D]
MUL X     ; ACC <- ACC * M[X]
ST  X     ; M[X] <- ACC
```

- Komutlar kısa ama 7 komut gerekir.
- Sıfır adresli: Bütün işlemler yığın (stack) üzerinde yapılır.

```
PUSH A
PUSH B
ADD         ; TOS <- TOS + TOS-1
PUSH C
PUSH D
ADD
MUL
POP X
```

Yığın işlemleri (kitaptaki model)

- Yığın işaretçisi (SP) her zaman dolu olan en üst gözü gösterir. Yığın düşük adreslere doğru büyür (örnekte SP = 101, dolu gözler 101, 102, 103...).
- PUSH: önce $SP \leftarrow SP - 1$, sonra $M[SP] \leftarrow R_1$.
- POP: önce $R_1 \leftarrow M[SP]$, sonra $SP \leftarrow SP + 1$.
- Ayrıntılar mimariye göre değişebilir (8086 farklı olabilir).
- Ters Lehçe gösterimi (Reverse Polish Notation) yığın kullanımına örnek olarak verildi.

Adresleme modları

- Örnek: Bellekte 250-251 adreslerinde 2 baytlık bir komut var. Opcode "load to AC". Adres alanında 500.
- Modlara göre davranış (tablo yukarıda):
  - Doğrudan: 500 adresine gidilir, 800 okunur.
  - Anlık: AC'ye doğrudan 500 yüklenir. Sayı komutun 251 adresindeki alanındadır.
  - Dolaylı: 500 adresindeki 800 bir adrestir. 800 adresindeki 300 yüklenir.
  - PC'ye göreli: Komut getirildikten sonra PC = 252. EA = 500 + 252 = 752.
  - İndisli: EA = adres alanı + indis register'ı = 500 + 400 = 900. 900 adresindeki 200 yüklenir.
  - Register: AC ← R1 = 400.
  - Register dolaylı: R1 = 400 bir adrestir. 400 adresindeki 700 yüklenir.
- Bir öğrenci güncel işlemcilerin bunları destekleyip desteklemediğini sordu. Hoca: "Fazlası var, eksiği yok." Hepsi aynı işlemcide olmak zorunda değil ama genel olarak bunları kapsar.

Komut türleri

Veri transfer komutları

- LD (load), ST (store), MOVE, XCHG (exchange), PUSH, POP, IN, OUT.
- XCHG tek adımda olmaz. Arka planda bir mikroprogram geçici bir alan kullanarak değişimi yapar.

G/Ç adresleme

- Bağımsız G/Ç (isolated I/O, independent I/O): Bellek ve G/Ç portlarının adres alanları ayrıdır. 8086 böyledir.
  - 8086'da 20 adres biti, 1 MB bellek alanı. Belleğe MOV ile erişilir.
  - G/Ç alanına IN ve OUT ile erişilir.
  - `IN AL, port` : port numarası 8 bit, G/Ç alanı 256 adres.
  - `IN AL, DX` : DX 16 bit, G/Ç alanı 2^{16} = 64K adres. Okunan veri yine 8 bittir.
  - G/Ç alanı anakartta bir bellek çipi değildir. Belirli adresleri çözen adres çözme devreleri vardır. Ayrıntısı mikroişlemciler dersinde.
- Belleğe eşlenmiş G/Ç (memory-mapped I/O): Tek bir adres alanı vardır. Bu alanın bir bölümü G/Ç portlarına ayrılır.

Veri işleme komutları

- Aritmetik: INC, DEC, ADD, SUB, MUL, DIV (her mimaride olmayabilir), ADDC (add with carry), SUBB (subtract with borrow), ters çıkarma ($B - A$), NEG.

- Lojik: CLR, SET, NOT (complement), AND, OR, XOR, CLRC, SETC, COMC.
- Kaydırma: lojik kaydırma, aritmetik kaydırma, döndürme (rotate), elde ile döndürme (rotate with carry).
  - Lojik kaydırmada dışarıdan giren bit 0'dır. Dışarı çıkan bit carry'ye gider.
  - Sağa kaydırma 2'ye bölmedir (aşağı yuvarlanır). Sola kaydırma 2 ile çarpmadır.
  - Aritmetik sağa kaydırmada işaret biti kendi üstüne yeniden yüklenir. İşaret korunur.
  - Aritmetik sola kaydırmada işaret biti taşabilir. Kaydırmadan önce R_7 ve R_6 farklıysa taşma olur: $V = R_7 \oplus R_6$.
  - Döndürmede dışarı çıkan bit öbür uçtan geri girer. Elde ile döndürmede carry de zincire katılır.

Program kontrol komutları

- Branch, Jump, CALL, RET, CMP, TEST.
- CMP R1, R2: $R1 - R2$ yapar, sonucu hiçbir register'a yazmaz, yalnız durum bitlerini günceller.
- TEST R1, R2: $R1 \wedge R2$ yapar, yalnız durum bitlerini günceller. Sonuç 0 ise Z = 1.
- Koşullu dallanma: BZ (Z = 1), BNZ (Z = 0), BC (C = 1), BN/BM (N = 1), BV (V = 1) gibi.

Karşılaştırma ve koşullu dallanma

- İşaretsiz sayılar (8 bit): 0-255. İşaret biti ve taşma yoktur. Yalnız C (borrow) ve Z kullanılır.
- İşaretli sayılar (8 bit, ikiye tümleyen): -128 ile +127. N, V ve Z kullanılır.
- CMP A, B'den sonra işaretli karşılaştırmada $A > B$ koşulu: $(N \oplus V) + Z = 0$.
  - $N = 0, V = 0$: Sonuç pozitif ve doğru. $A \geq B$ (Z = 0 ise $A > B$).
  - $N = 0, V = 1$: Görünen işaret yanlış, gerçek sonuç negatif. $A < B$.
  - $N = 1, V = 0$: Sonuç negatif ve doğru. $A < B$.
  - $N = 1, V = 1$: Görünen işaret yanlış, gerçek sonuç pozitif. $A > B$.
- İşaretsiz karşılaştırmada (C = borrow):
  - $A > B$ (branch if above): $C + Z = 0$.
  - $A \geq B$: $C = 0$.
  - $A < B$: $C = 1$.

Örnek 1: İşaretsiz $A = 93$, $B = 92$

- $A - B = A + (B$'nin ikiye tümleyeni$)$. Sonuç 00000001 = 1, carry = 1.
- Carry 1 olduğu için borrow = 0. Çıkarmada ödünç alınmadı.
- Z = 0.
- Dallanmalar: $A > B$ doğru, $A \geq B$ doğru, $A < B$ yanlış, $A \leq B$ yanlış, $A = B$ yanlış.

Örnek 2 (eski sınav sorusu)

- (a) 8 bitlik işaretli sayılarla aritmetik kaydırma:

Sayı	İkili	ASL	V	ASR
+62	00111110	01111100 (+124)	0	00011111 (+31)
-75	10110101	01101010 (+106, yanlış)	1	11011010 (-38)

- -75: Önce $+75 = 01001011$ yazılır, sonra ikiye tümleyeni alınır: 10110101.
- -75 için $R_7 = 1$, $R_6 = 0$. Farklı oldukları için ASL taşma üretir.
- ASR'de taşma olmaz. $-75/2 = -37{,}5$, aşağı yuvarlanır: -38.
- (b) İşaretli $A - B$:
  - $A = -34$ (11011110), $B = +94$:
    - $A + (-B) = 11011110 + 10100010 = 10000000 = -128$.
    - İşaret bitine giren elde 1, çıkan elde 1. $V = 1 \oplus 1 = 0$. Taşma yok; -128 8 bite sığar.
    - $N = 1$, $Z = 0$. $A > B$ koşulu: $(1 \oplus 0) + 0 = 1$, yanlış. Doğru: $-34 < 94$.

- $A = +82$ (01010010), $B = -46$:
  * $A + (-B) = 01010010 + 00101110 = 10000000$. Gerçek sonuç $+128$, 8 bite sığmaz.
  * İşaret bitine giren elde 1, çıkan elde 0. $V = 1$. $N = 1$, $Z = 0$.
  * İki pozitif sayı toplandı ama sonuç negatif görünüyor. Bu taşmadır.

Ders sonrası notlar

- **RISC/CISC karışıklığı:** Hoca karşılaştırırken bir yerde "CISC tarafında adresleme modlarının sayısı sınırlı ve komut uzunlukları sabit" dedi. Bu RISC'in özelliğidir. CISC'te adresleme modu çok, komut uzunluğu değişkendir.
- **PC'ye göreli mod:** AC'ye yüklenen değer altyazıda geçmiyor. Tablodaki 600 değerini kitaptan kontrol edin.
- **Takas:** Bir öğrenci ikinci transferin sonraki periyotta yapılıp yapılamayacağını sordu, hoca "tabii ki" dedi. İki transfer ayrı periyotlarda yapılabilir, ama sonuç takas olmaz. İki register'da da R_1'in eski değeri kalır. Python ile denendi. Takas için geçici alan gerekir (bkz. XCHG).
- **PUSH'ta SP:** Sıfır adresli örnekte hoca "pointer bir artıyor, bir üst gözü gösteriyor" dedi. Yukarıdaki modelde yığın düşük adreslere büyür. PUSH'ta SP azalır.
- **Sıfır adresli örnek:** Hoca yalnız ilk adımları anlattı. Yukarıdaki tam program kitaptaki örnekten tamamlandı.
- **İkinci işaretli örnek:** Hoca $A > B$ için "false" sonucunu ilk sayı çifti (-34, 94) için söyledi. İkinci çift (82, -46) için: $N = 1$, $V = 1$, $Z = 0$, $(N \oplus V) + Z = 0$. Yani $A > B$ doğrudur ($82 > -46$). Taşma olsa bile karşılaştırma doğru sonuç verir. Sonuçlar Python ile doğrulandı.
- **Lojik sola kaydırma yönü:** Hoca "sola kaydırmada da seri giriş 0" dedi. Doğru: lojik sola kaydırmada en düşük bite 0 girer, en yüksek bit carry'ye gider.
- **Bir öğrencinin onluk tabanda tümleyen benzetmesi:** $4 - 2$ için 2'nin 10'a tümleyeni 8; $4 + 8 = 12$, elde atılınca sonuç 2. Benzetme doğrudur. 8 bit ikili sayıda tümleme $2^8 = 256$'ya göre yapılır.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.